ShopEz

E-Commerce Clothing Application

Full Stack (MERN) Project Documentation

Prepared by: Turlapati Lokesh
Anurag University –Cyber Security
GitHub: Lokesh-Turlapati/VIP_C2_ShopEz

1. Introduction

Project Title

ShopEz – E-Commerce Clothing Application

Team Members

This project was developed individually by Turlapati Lokesh, who was responsible for end-to-end design and development of the client storefront, admin panel, and backend API.

2. Project Overview

Purpose

ShopEz is a full-stack e-commerce web application built for buying clothing across three categories — Men, Women, and Kids. The goal of the project is to demonstrate a complete MERN-based online shopping workflow: browsing categorized products, managing a shopping cart, and maintaining the product catalog through a dedicated admin panel — mirroring the core functionality of a real-world retail storefront.

Key Features

- Category-based storefront with dedicated Men, Women, and Kids product listings
- Homepage banner and promotional sections to highlight collections and offers
- Time-bound discount banners (e.g. "Flat 50% OFF") on category pages
- Product grid with original price, discounted price, and discount percentage badges
- Shopping cart with quantity, subtotal, shipping fee, promo code entry, and order total
- Dedicated Admin Panel to add new products and manage the existing product catalog
- Product image upload support from the admin interface
- Orders section for tracking placed orders
- Search bar for quick product lookup

3. Architecture

Frontend

The frontend is built using React (Vite), split into two independent applications:

- Client App – the customer-facing storefront (Shop, Men, Women, Kids, Cart, Orders, Login) served on localhost:3000.
- Admin App – a separate React application for catalog management (Add Product, Product List) served on localhost:3001.

Both apps are organized using a component-based architecture, with reusable UI components (navbar, product cards, banners) kept separate from route-level page components, and client-side routing used to navigate between category pages without full page reloads.

Backend

The backend is built with Node.js and Express.js, exposing a REST API consumed by both the client and admin applications. It handles product CRUD operations, cart/order processing, and image uploads, and serves uploaded product images as static assets.

Database

MongoDB (with Mongoose) is used as the data layer. The core schema design includes:

- Product — title, description, price, offerPrice, category (men/women/kids), image, createdAt

- Order — user reference, ordered items, quantities, total amount, status, timestamps
- User — name, email, password (hashed), role (customer/admin) for authentication

The Product schema drives both the storefront listings and the admin product table, keeping a single source of truth for catalog data.

4. Setup Instructions

Prerequisites

- Node.js (v18 or later) and npm
- MongoDB (local instance or a MongoDB Atlas cluster)
- Git

Installation

- Clone the repository: git clone https://github.com/Lokesh-Turlapati/VIP_C2_ShopEz.git
- Install server dependencies: cd server && npm install
- Install client dependencies: cd client && npm install
- Install admin dependencies: cd admin && npm install
- Create a .env file inside the server directory with MONGO_URI, PORT, and JWT_SECRET, and a .env file inside client/admin with the API base URL (e.g. VITE_BACKEND_URL)
- Start MongoDB (if running locally) before starting the server

5. Folder Structure

Admin

- node_modules — installed dependencies
- public — static assets and index.html
- src — admin application source (Add Product form, Product List view, shared components)

Client

- build — production build output
- node_modules — installed dependencies
- public — static assets and index.html
- src / components — reusable UI components (navbar, product card, banner, footer)
- src / pages — route-level pages (Home, Men, Women, Kids, Cart, Orders, Login)

Server

- node_modules — installed dependencies
- images — uploaded product images
- admin — admin-specific route/controller logic
- public — publicly served static files

6. Running the Application

Each part of the application is run independently, typically in three separate terminals:

- Backend: cd server && npm start — runs the Express API server
- Client: cd client && npm start — runs the customer storefront on localhost:3000
- Admin: cd admin && npm start — runs the admin panel on localhost:3001

The client and admin apps communicate with the backend over the REST API using the configured backend URL environment variable.

7. API Documentation

The backend exposes REST endpoints for products, cart, orders, and authentication. Representative endpoints are summarized below.

Product Endpoints

Method	Endpoint	Description	Access
GET	/api/products	Get all products (optional ?category= filter)	Public
GET	/api/products/:id	Get details of a single product	Public
POST	/api/products	Add a new product (title, description, price, offerPrice, category, image)	Admin
DELETE	/api/products/:id	Remove a product from the catalog	Admin

Cart & Order Endpoints

Method	Endpoint	Description	Access
GET	/api/cart	Get the current user's cart contents	User
POST	/api/cart/add	Add an item to the cart	User
PUT	/api/cart/update	Update the quantity of a cart item	User
DELETE	/api/cart/remove/:id	Remove an item from the cart	User
POST	/api/orders	Place a new order from the current cart	User
GET	/api/orders	Get order history	User/Admin

Authentication Endpoints

Method	Endpoint	Description	Access
POST	/api/auth/register	Register a new user account	Public
POST	/api/auth/login	Authenticate a user and return a JWT	Public

Example response for GET /api/products:

```
[ { "_id": "665f...", "title": "High Waist Skinny Denim Jeans", "price": 109,
"offerPrice": 74, "category": "women", "image": "/images/..." } ]
```

8. Authentication

User accounts and admin access are handled separately. On the client, a Login option is available in the navbar for customers to sign in before checking out. On the admin side, access to catalog-management actions (adding or removing products) is restricted to authenticated admin users.

- Authentication is token-based, using JSON Web Tokens (JWT) issued at login and validated on protected routes.
- Passwords are hashed (e.g. using bcrypt) before being stored in the database.

- Protected endpoints (such as product creation/removal) expect the JWT in the request Authorization header and verify it via Express middleware before allowing the operation.

9. User Interface

The storefront presents a clean, category-driven shopping experience, while the admin panel offers a simple dashboard for catalog management. Key screens are summarized below; full screenshots are provided in Section 11 (Screenshots or Demo).

- Home — hero banner introducing the ShopEz brand with a call-to-action to shop the collection
- Men / Women / Kids — category pages with a countdown discount banner and a paginated product grid showing original price, discounted price, and discount percentage
- Cart — itemized cart view with product, price, quantity, total, promo code entry, and a checkout action
- Admin – Add Product — a form to enter product title, description, price, offer price, category, and upload a product image
- Admin – Product List — a management table listing all products with old price, new price, category, and a remove action

10. Testing

Testing was carried out primarily through manual functional testing during development:

- Manual UI testing across the Home, Men, Women, Kids, and Cart pages to verify navigation, product rendering, and pricing display
- Manual verification of admin workflows — adding a product from the Add Product form and confirming it appears correctly in the Product List
- API testing of backend endpoints using Postman/Thunder Client to verify request/response behavior for product CRUD operations
- Cross-checking that discount badges and offer prices calculate and display correctly against the base price

Automated unit/integration testing (e.g. Jest, React Testing Library, Supertest) is identified as a future enhancement, see Section 13.

11. Screenshots or Demo

The following screenshots demonstrate the ShopEz storefront and admin panel in action.

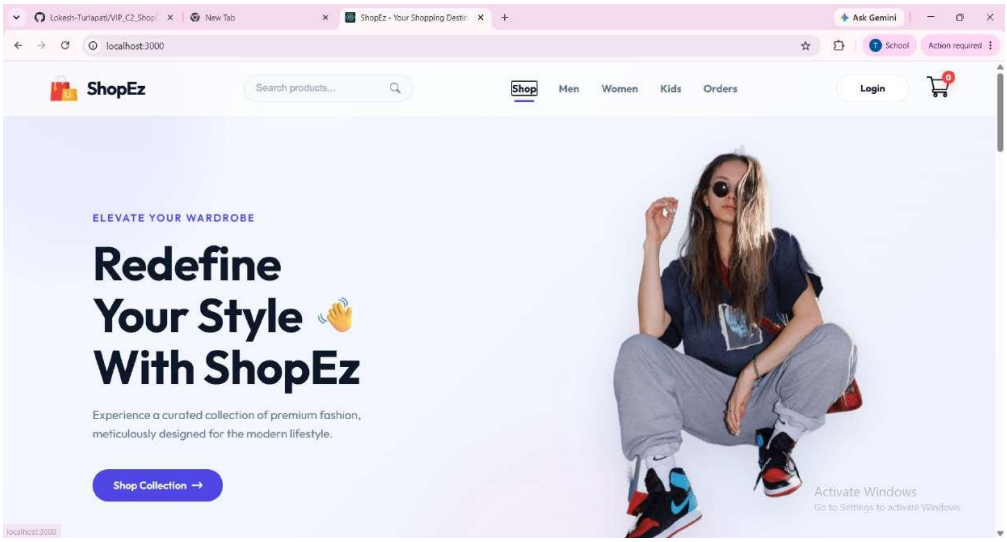

Fig 1: Home page — hero banner and call-to-action

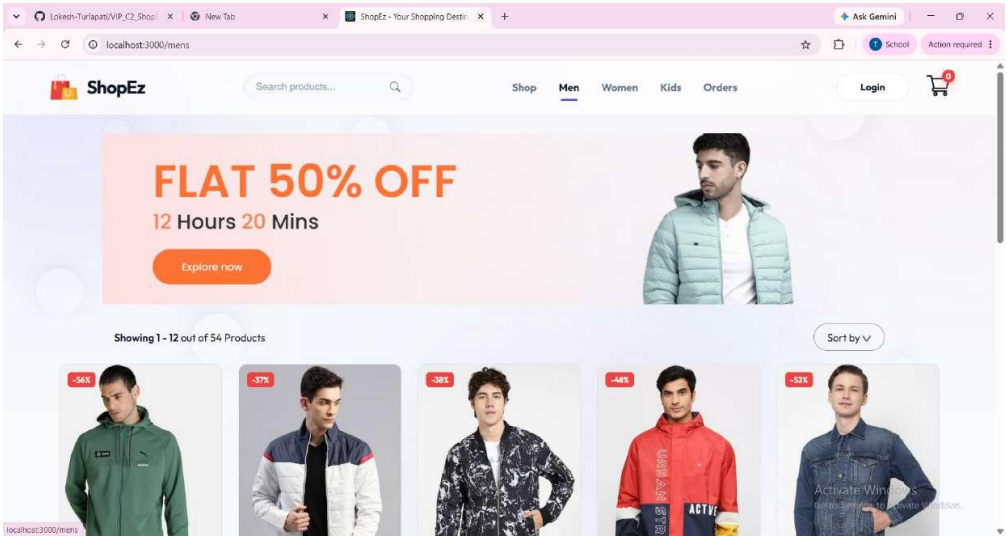

Fig 2: Men category page with discount banner and product grid

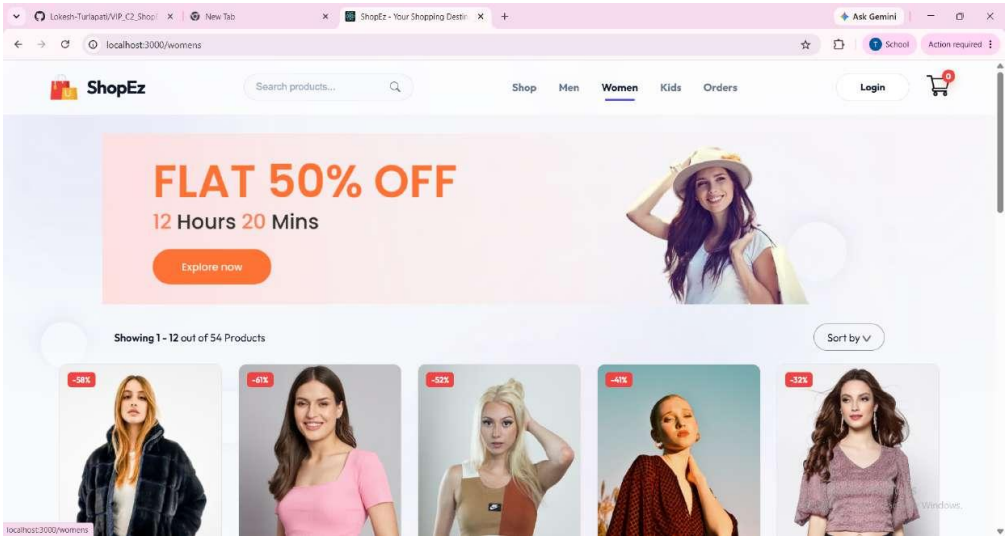

Fig 3: Women category page with discount banner and product grid

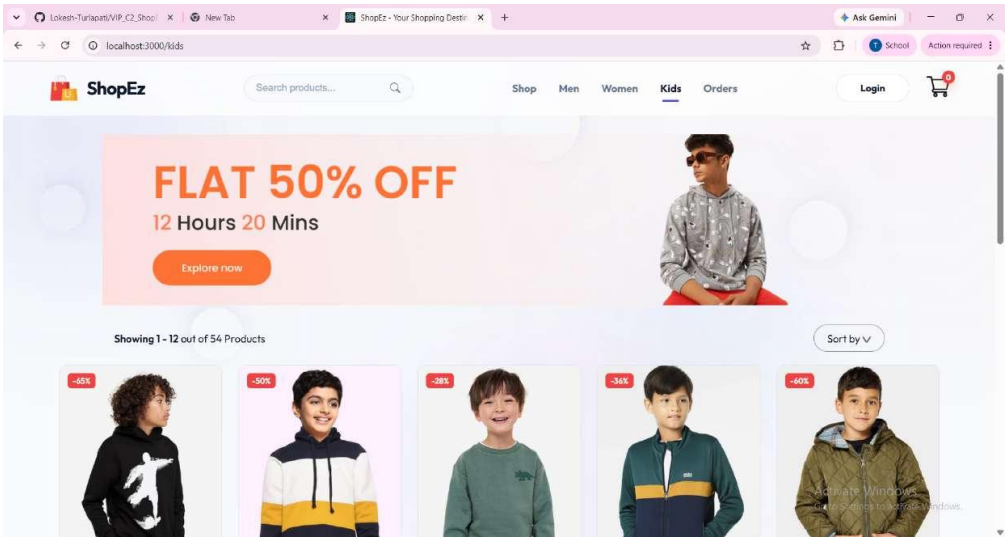

Fig 4: Kids category page with discount banner and product grid

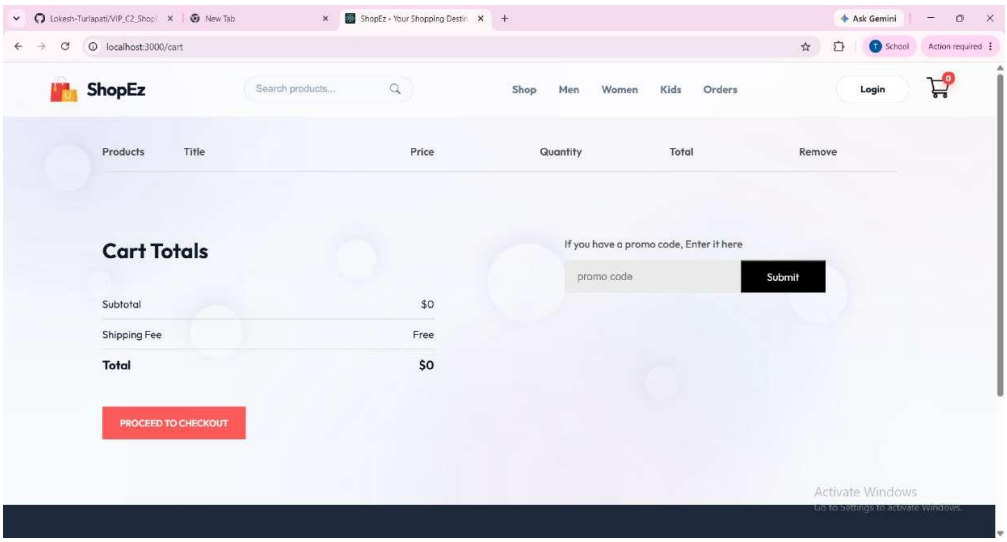

Fig 5: Shopping cart with totals and promo code entry

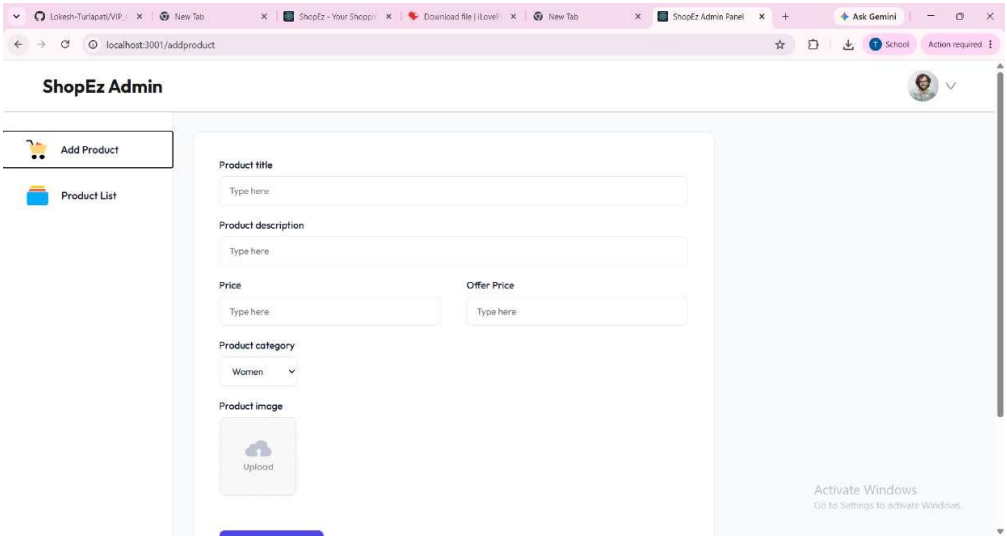

Fig 6: Admin panel — Add Product form

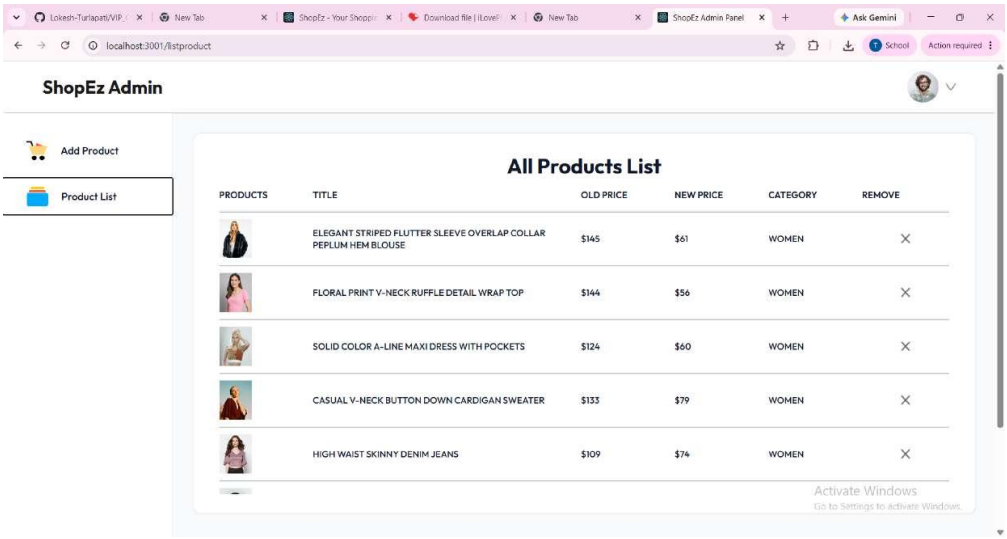

Fig 7: Admin panel — All Products List with remove action

12. Known Issues

- Promo code field on the cart page is currently a placeholder and does not apply real discounts
- Checkout does not yet integrate with a payment gateway; "Proceed to Checkout" does not complete a real transaction
- User login on the client is present in the UI but not fully wired to persisted authentication state across sessions
- Product images are stored locally on the server rather than on a cloud storage/CDN service
- Order history and order-status updates are not yet reflected in the client Orders page
- No pagination controls beyond the default product count on category pages
- Limited responsive design testing on smaller mobile screen sizes

13. Future Enhancements

- Integrate a real payment gateway (e.g. Razorpay/Stripe) for end-to-end checkout
- Persist user authentication with refresh tokens and protected client routes
- Add product search and filter/sort options (price, category, size, popularity)
- Add a wishlist feature and product ratings/reviews
- Move image storage to a cloud service (e.g. Cloudinary/AWS S3)
- Build out order tracking and status updates visible to both customer and admin
- Add automated unit and integration tests for both frontend and backend
- Improve responsive design for mobile and tablet breakpoints
- Add an admin analytics dashboard (sales, top products, inventory alerts)